DATA ANALYTICS FOR BUSINESS

with Lizhen Xu, Ph.D.

Neural Networks

Learning Objectives

- Describe the structure of single-layer neural networks
- Describe the structure of multi-layer neural networks
- Estimate a neural network

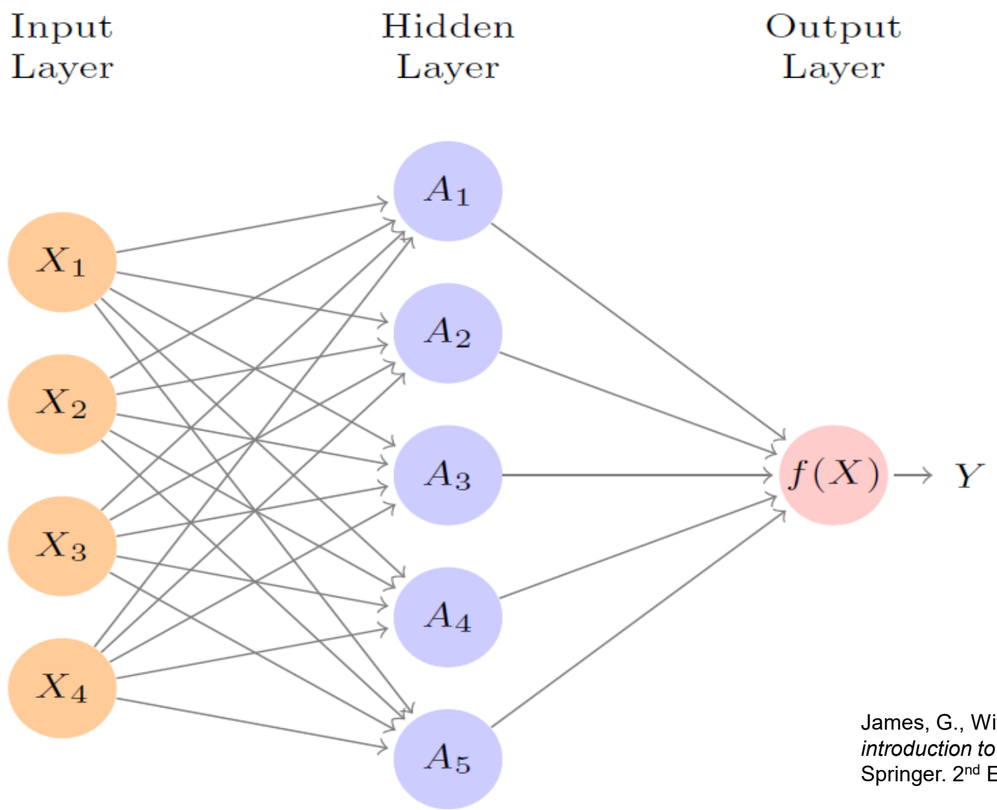
Neural Networks

- Neural Networks: History
  - First introduced in the late 1980s
  - Resurfaced after 2010, with the new name *Deep Learning*
  - Success stories in image classification, speech and text modeling
  - Benefit from the availability of extremely large training datasets

- Neural Network Models:
  - Neural Networks also called **Artificial Neural Networks** (**ANN**)
  - Feed-forward Neural Networks
  - Convolutional Neural Networks (CNN)
  - Recurrent Neural Networks (RNN)

Single Layer Neural Networks (1/4)

- Feed-forward Neural Networks
  - Information moves in *only one direction*—*forward*—from the *input nodes*, through the *hidden nodes*, and to the *output nodes*



James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: with applications in R*. Springer. 2nd Ed. p 405.

Single Layer Neural Networks (2/4)

- Feed-forward Neural Networks
  - *Inputs*: $X = (X_1, X_2, \dots, X_p)$
  - *Activations* (hidden nodes): $A_1, A_2, \dots, A_K$
  - *Output*: $Y = f(X)$

$$A_k = h_k(X) = g(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j)$$

$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

  - $\boldsymbol{g}(\cdot)$: Activation function, non-linear function
  - The *weights* (i.e., coefficients w_{kj}) and the *bias* terms (i.e., intercept w_{k0}), are <u>learned (or estimated) from the data</u>

  - Minimize the *loss function*: squared-error

$$\sum_{i=1}^{n} (y_i - f(X_i))^2$$

Single Layer Neural Networks (3/4)

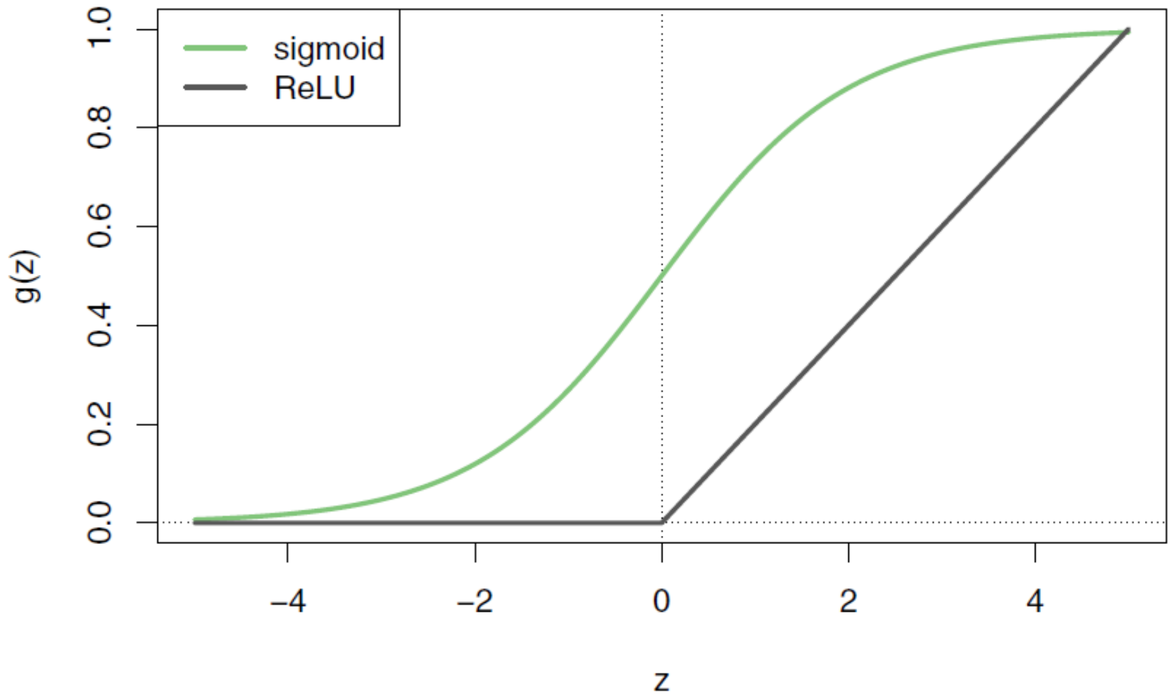
- Choice of Activation Functions
  - *Sigmoid* activation function

$$g(z) = \frac{e^z}{1 + e^z}$$

  - *ReLU* (rectified linear unit) activation function

$$g(z) = (z)_+ = \begin{cases} 0, & z < 0 \\ z, & z \geq 0 \end{cases}$$

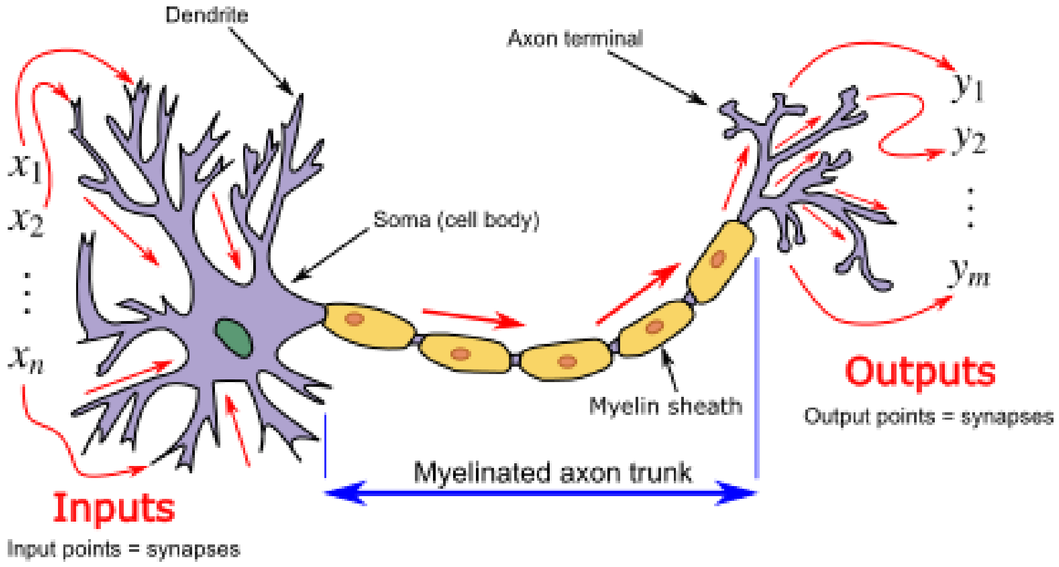
  - Nonlinearity of $g(\cdot)$ is essential
    - Otherwise would collapse into a simple linear model
  - Allow the model to capture complex nonlinear and interaction effects



James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: with applications in R*. Springer. 2nd Ed. p 406.

Single Layer Neural Networks (4/4)

- Analogous to neurons
  - Activations (hidden nodes) are like neurons in the brain
  - Can “fire” signals to interconnected neurons
  - Move information from inputs to outputs



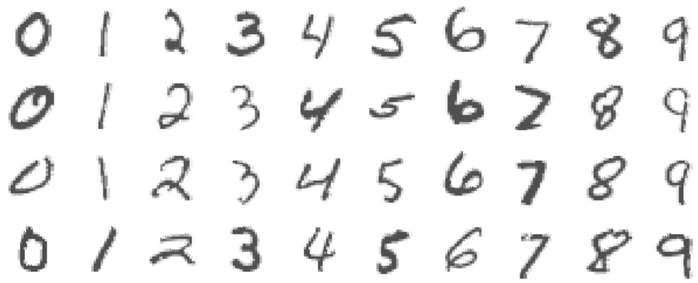
Source: Egm4313.s12 at English Wikipedia

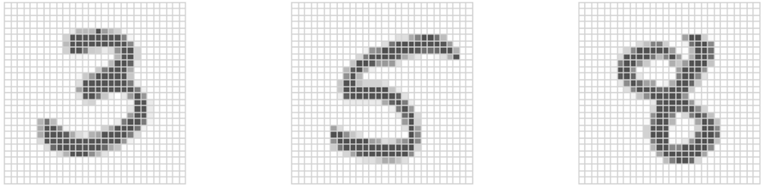
- Connection to Linear/Logistic Regressions
  - Linear/Logistic Regressions can be viewed as special cases of neural networks
  - No hidden layer

Multi-layer Neural Networks (1/4)

- Multi-layer Neural Networks
  - In theory, a single hidden layer with *a large number of* units can approximate most functions
  - In practice, it's easier to learn with *multiple layers*, each of a *modest size*

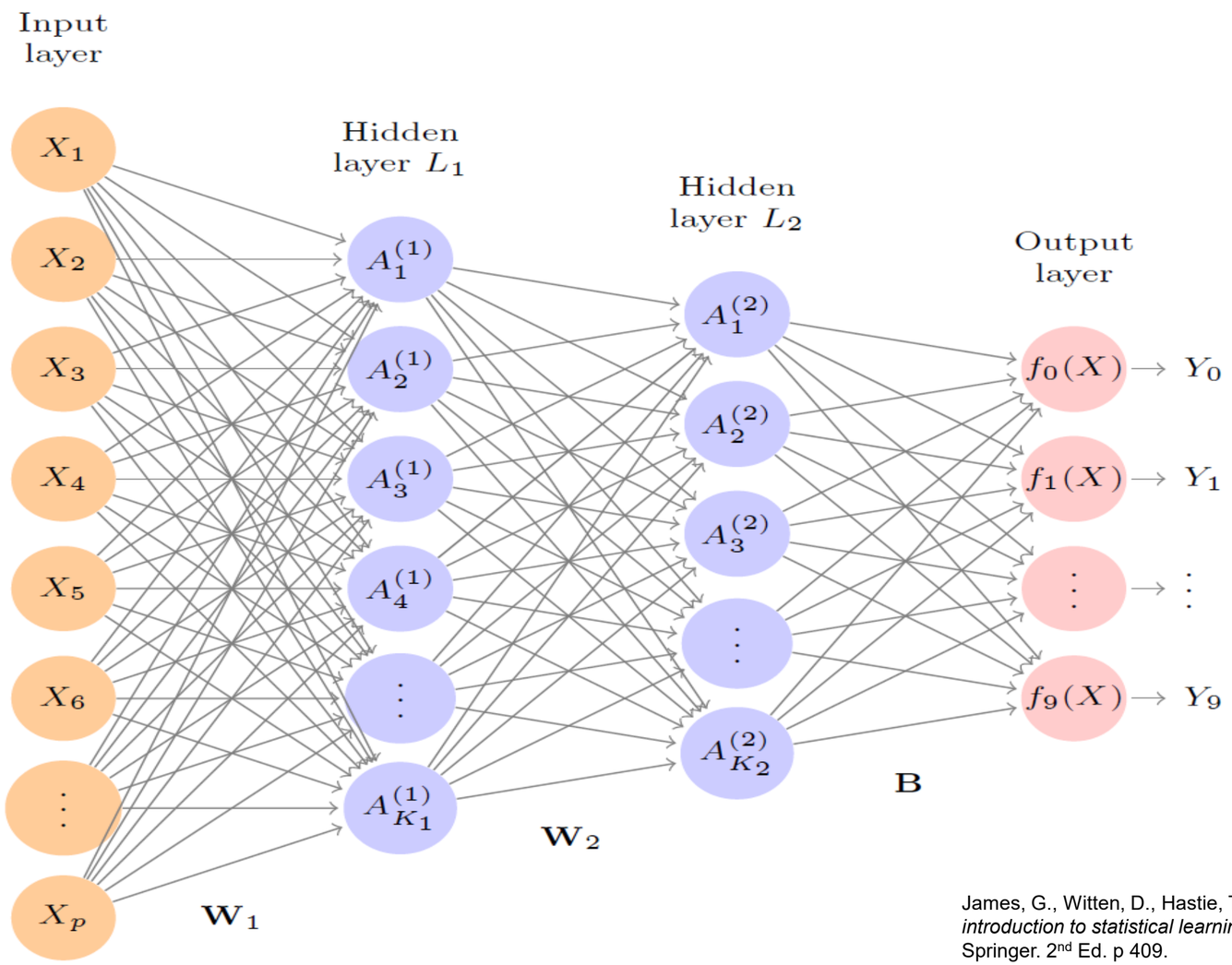
- The MNIST handwritten digit dataset
  - Each grayscale image has 28x28 pixels
  - Each pixel is an 8-bit number (0-255), representing the darkness level
  - Outputs: the recognized digit class: 0-9





James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: with applications in R*. Springer. 2nd Ed. p 407.

Multi-layer Neural Networks (2/4)



James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: with applications in R.* Springer. 2nd Ed. p 409.

Multi-layer Neural Networks (3/4)

- Multi-layer Neural Networks
  - *Activations* in the previous hidden layer become the *inputs* for the next hidden layer

$$A_k^{(1)} = h_k^{(1)}(X) = g(w_{k0}^{(1)} + \sum_{j=1}^{p} w_{kj}^{(1)} \boldsymbol{X_j})$$

$$A_l^{(2)} = h_l^{(2)}(X) = g(w_{l0}^{(2)} + \sum_{k=1}^{K_1} w_{lk}^{(2)} \boldsymbol{A_k^{(1)}})$$

$$Z_m = \beta_{m0} + \sum_{l=1}^{K_2} \beta_{ml} \boldsymbol{A_l^{(2)}}$$

  - At the output layer, we use the *softmax* activation function

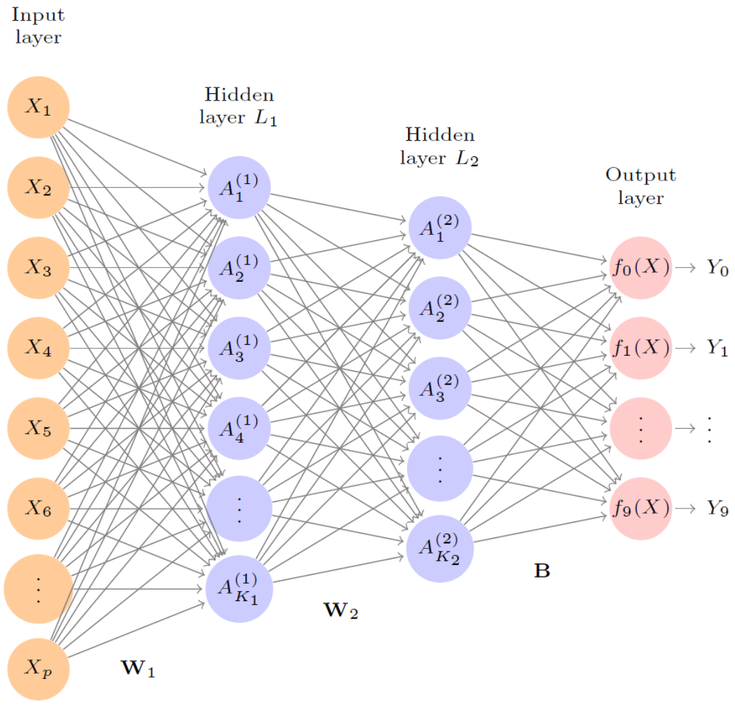
$$f_m(X) = \Pr(Y = m|X) = \frac{e^{Z_m}}{\sum_{l=0}^{9} e^{Z_l}}$$

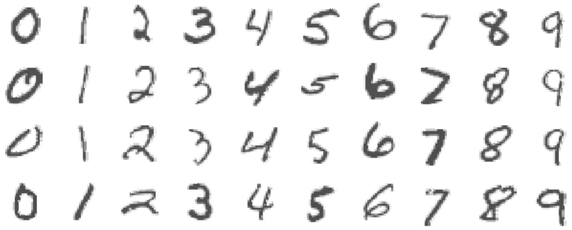
  - Minimize the negative log-likelihood:
    - A.k.a. "*cross-entropy*"

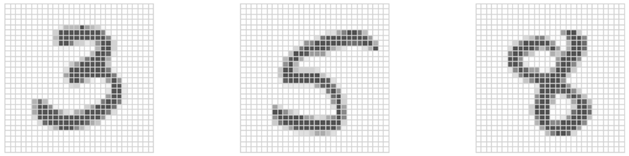
$$-\sum_{i=1}^{n} \sum_{m=0}^{9} y_{im} \log(f_m(X_i))$$

Multi-layer Neural Networks (4/4)

- A Large Number of Parameters
  - Input nodes: 28 x 28 + 1 = 785
  - First hidden layer: 256 units
  - Second hidden layer: 128 units
  - Output nodes: 10
  - First matrix: 785 x 256 = 200,960 elements
  - Second matrix: 257 x 128 = 32,896 elements
  - Third matrix: 129 x 10 = 1,290 elements
  - Totally 235,146 parameters!
  - More parameters than the observations!







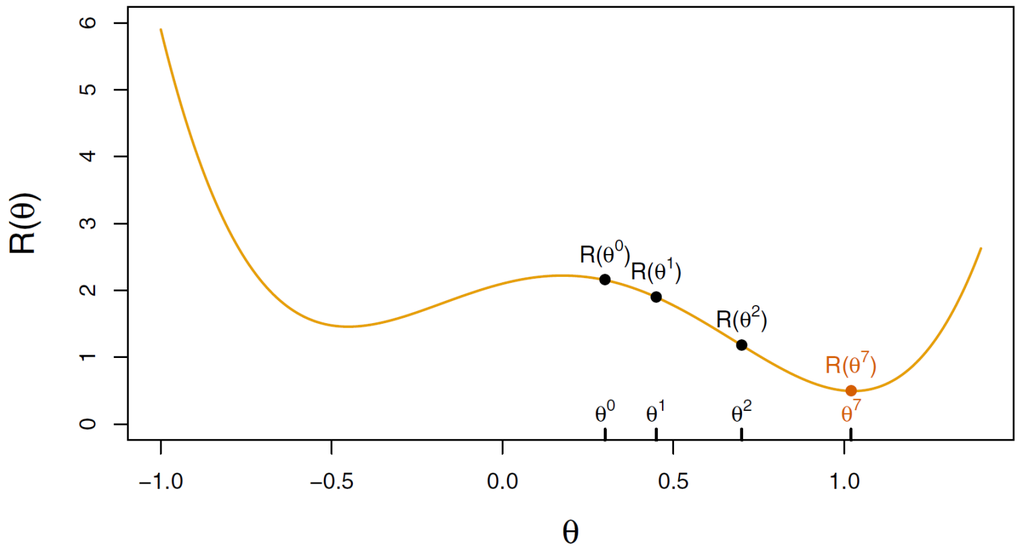
Estimating a Neural Network

- Objective function:

$$\min \sum_{i=1}^{n} (y_i - f(X_i))^2$$

$$\min - \sum_{i=1}^{n} \sum_{m=0}^{9} y_{im} \log(f_m(X_i))$$

  - Nonconvex in parameters
  - Multiple solutions
  - Local minimum vs. global minimum



James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: with applications in R.* Springer. 2nd Ed. p 435.

Gradient Descent

- Objective function
  - Slow learning: Gradient Descent
  - Regularization
- Gradient Descent

$$\theta^{m+1} = \theta^{m} - \rho \nabla R(\theta^{m})$$

  - $\nabla R(\theta^{m}) = \left.\frac{\partial R(\theta)}{\partial \theta}\right|_{\theta=\theta^{m}}$ gives the direction in θ-space in which the objective function $R(\theta)$ increases the most rapidly.
  - ρ: the *learning rate*
  - Error backpropagation: the gradient takes a simple form following the *chain rule* of differentiation
  - *Case Updating* vs *Batch Updating*
    - Update the weights after each record (vs all records) is (are) run through the network
    - *Minibatch:* randomly sample a small fraction of records to compute a gradient step
    - Epoch: an equivalent of all training data have been processed

Regularization

- Ridge Regularization
  - Add penalty term to penalize the number of parameters

$$\min \sum_{i=1}^{n} (y_i - f(X_i))^2 + \lambda \sum_j \theta_j^2$$

  - λ: strength of regularization (the larger, the fewer parameters)

- Dropout Learning
  - Randomly remove a fraction ϕ of the nodes in a layer when fitting the model
  - Done separately each time a training observation is processed
  - Borrow the idea from random forest
  - One form of regularization

Tuning the Network

- Neural networks require a lot of fine tuning and tinkering
  - A lot of choices to make that will affect the learning process/outcome
- *Hyperparameters*
  - Not the model parameters to be estimated from the data; but their values affect the learning process/outcome; need to be set before training
  - Model Hyperparameters
    - Number of hidden layers
    - Number of activation nodes in a layer
    - Activation functions
  - Algorithm Hyperparameters
    - Learning rate
    - Strength of regularization
    - Dropout rate
    - Batch size
    - Number of epochs
    - ……

Summary

- Single-layer neural networks
- Multi-layer neural networks
- Estimating a Neural Network
  - Gradient descent
  - Regularization
  - Droupout learning